

Delibera — обновление до .NET 10 и C# 15

Документ описывает изменения, внесённые при переводе проекта **Delibera** на платформу **.NET 10** и язык **C# 15 (preview)**, оптимизации производительности и новые примеры работы роли **Operator** с MCP-инструментами (браузер и генерация Map-презентаций).

Все изменения выполнены в ветке `feature/operator-mcp-role`. Проект собирается командой `dotnet build` без предупреждений и ошибок.

Содержание

- 1. [Целевая платформа и версия языка](#)
- 2. [Обновление зависимостей](#)
- 3. [Оптимизации производительности \(high performance\)](#)
- 4. [Примеры Operator + MCP-инструменты](#)
- 5. [Сборка и запуск](#)
- 6. [Краткая сводка изменённых файлов](#)

1. Целевая платформа и версия языка

Оба проекта переведены на единый стек:

Свойство	Было	Стало
TargetFramework	net10.0	net10.0
LangVersion	14.0	preview (C# 15)
Nullable	enable	enable
ImplicitUsings	enable	enable

Изменения внесены в файлы:

- `src/Delibera.Core/Delibera.Core.csproj`
- `src/Delibera.ConsoleApp/Delibera.ConsoleApp.csproj`

Важно про «C# 15»

На момент обновления установлен GA-релиз **.NET SDK 10.0.301**, в котором компилятор Roslyn поддерживает версии языка до **14.0** включительно, а следующая (будущая C# 15) доступна **только** через значение `preview`. Прямое указание `<LangVersion>15.0</LangVersion>` приводит к ошибке компиляции:

```
error CS1617: Invalid option '15.0' for /langversion
```

Поэтому используется `<LangVersion>preview</LangVersion>` — это включает максимально доступный набор языковых возможностей следующей версии (C# 15). Когда выйдет SDK с официальной поддержкой `15.0`, значение можно будет заменить на `15.0` без других изменений в коде.

2. Обновление зависимостей

Все NuGet-пакеты приведены к версиям, совместимым с .NET 10. Большинство пакетов уже находились на актуальных версиях для .NET 10; обновлён `Microsoft.SourceLink.GitHub`.

Delibera.Core

Пакет	Версия	Назначение
Microsoft.Extensions.Configuration.Abstractions	10.0.9	Конфигурация (DI)
Microsoft.Extensions.Configuration.Json	10.0.9	Чтение appsettings.json
Microsoft.Extensions.DependencyInjection	10.0.9	Контейнер DI
Microsoft.Extensions.DependencyInjection.Abstractions	10.0.9	Абстракции DI
Microsoft.Extensions.Options	10.0.9	Паттерн Options
Microsoft.Extensions.Options.ConfigurationExtensions	10.0.9	Привязка Options к конфигурации
ModelContextProtocol	1.4.0	SDK для MCP-серверов (роль Operator)
Npgsql	10.0.3	PostgreSQL / pgvector
OllamaSharp	5.4.25	LLM-провайдер Ollama
Pgvector	0.3.2	Векторный поиск в PostgreSQL
Qdrant.Client	1.18.1	Векторное хранилище Qdrant
Microsoft.SourceLink.GitHub	10.0.300 (было 10.0.102)	Source Link для отладки

Delibera.ConsoleApp

Пакет	Версия	Назначение
<code>Microsoft.Extensions.Configuration</code>	10.0.9	Конфигурация
<code>Microsoft.Extensions.Configuration.Binder</code>	10.0.9	Привязка конфигурации
<code>Microsoft.Extensions.Configuration.Json</code>	10.0.9	JSON-конфигурация
<code>Microsoft.Extensions.Configuration.UserSecrets</code>	10.0.9	User Secrets
<code>Microsoft.Extensions.DependencyInjection</code>	10.0.9	Контейнер DI

Проверка актуальности выполнена через `dotnet list package --outdated` и NuGet flat-container API — на момент обновления более свежих стабильных версий для перечисленных пакетов нет.

3. Оптимизации производительности (high performance)

Внесены целенаправленные оптимизации «горячих» путей с применением `Span<T>` / `ReadOnlySpan<T>`, SIMD (`System.Numerics.Vector<T>`) и `ArrayPool<T>`. Все оптимизации сохраняют прежнее поведение (проверено численно).

3.1. SIMD-векторизация косинусной близости

Файл: `src/Delibera.Core/Compression/SemanticCompressor.cs` → `CosineSimilarity`

Косинусная близость — самый «горячий» цикл при семантическом сжатии и дедупликации (вызывается для каждой пары предложений по всем измерениям эмбединга, обычно 768–1536). Реализация переписана:

- Сигнатура изменена с `float[]` на `ReadOnlySpan<float>` — это zero-copy и позволяет передавать

массивы, срезы и `stackalloc`-буферы без аллокаций (вызывающий код с `float[]` работает без изменений).

- Основной цикл векторизован через `Vector<float>`: за одну итерацию обрабатывается `Vector<float>.Count` элементов (на современных x64/ARM64 — 8-16 значений за такт).
- Скалярный «хвост» обрабатывает остаток, а также служит запасным путём при отсутствии аппаратного ускорения (`Vector.IsHardwareAccelerated == false`).

```
internal static double CosineSimilarity(ReadOnlySpan<float> a, ReadOnlySpan<float> b)
{
    if (a.Length != b.Length || a.IsEmpty) return 0;

    float dot = 0, magA = 0, magB = 0;
    var i = 0;

    if (Vector.IsHardwareAccelerated && a.Length >= Vector<float>.Count)
    {
        var dotAcc = Vector<float>.Zero;
        var magAAcc = Vector<float>.Zero;
        var magBAcc = Vector<float>.Zero;
        var width = Vector<float>.Count;

        for (; i <= a.Length - width; i += width)
        {
            var va = new Vector<float>(a.Slice(i, width));
            var vb = new Vector<float>(b.Slice(i, width));
            dotAcc += va * vb;
            magAAcc += va * va;
            magBAcc += vb * vb;
        }

        dot = Vector.Dot(dotAcc, Vector<float>.One);
        magA = Vector.Dot(magAAcc, Vector<float>.One);
        magB = Vector.Dot(magBAcc, Vector<float>.One);
    }

    for (; i < a.Length; i++) { dot += a[i]*b[i]; magA += a[i]*a[i]; magB += b[i]*b[i]; }

    var denom = Math.Sqrt(magA) * Math.Sqrt(magB);
    return denom > 0 ? dot / denom : 0;
}
```

Корректность. Численное сравнение с эталонной (скалярной) реализацией на длинах 1, 3, 7, 8, 16, 17, 100, 1536 дало максимальную абсолютную погрешность $\approx 6 \cdot 10^{-8}$ (на уровне точности `float`), результат идентичен.

3.2. Подсчёт токенов без аллокаций (`ReadOnlySpan<char>`)

Файл: `src/Delibera.Core/Compression/TokenCounter.cs`

- Добавлена перегрузка `EstimateTokens(ReadOnlySpan<char> text)` — позволяет оценивать токены для срезов строк без выделения подстрок.
- Внутренний `CountWords` принимает `ReadOnlySpan<char>` и перебирает символы по `ref struct enumerator'y` — без боксинга и без копий.

- Существующий `EstimateTokens(string?)` сохранён и теперь делегирует в `span`-перегрузку (через `text.AsSpan()`), поэтому публичный API полностью обратно совместим.

3.3. Хеширование ключа кэша через `ArrayPool<T>` + `stackalloc`

Файл: `src/Delibera.Core/Compression/CompressionCache.cs` → `ComputeKey`

Раньше построение ключа кэша создавало две лишние кучи-аллокации на каждый вызов: интерполированную строку `"{strategy}:{text}"` и массив байтов из `Encoding.UTF8.GetBytes`. Новая реализация:

- Арендует буфер символов из `ArrayPool<char>.Shared`, склеивает в него `strategy + ':' + text`.
- Арендует буфер байтов из `ArrayPool<byte>.Shared` и кодирует UTF-8 напрямую в него (`Encoding.UTF8.GetBytes(span, span)`).
- Считает SHA-256 в стек-буфер `stackalloc byte[SHA256.HashSizeInBytes]` и форматирует hex.
- Гарантированно возвращает арендованные буферы в `finally`.

Итог: на «тёплом» пути кэширования сжатия исключены промежуточные аллокации больших строк/массивов, что снижает нагрузку на GC при больших объёмах контекста дебатов.

3.4. `ValueTask` в роли `Operator`

Метод освобождения ресурсов `Operator` реализован как `public async ValueTask DisposeAsync()` (`src/Delibera.Core/Council/Operator.cs`) — стандартный для .NET 10 паттерн `IAsyncDisposable`, позволяющий избежать аллокации `Task` на пути освобождения MCP-клиентов. Новый пример (`OperatorMcpToolsExample`) использует `await using`, корректно задействуя этот путь.

Сводка оптимизаций

Приём	Где применён	Эффект
<code>ReadOnlySpan<T></code> + SIMD	<code>SemanticCompressor.CosineSimilarity</code>	8–16× ширина обработки, без копий
<code>ReadOnlySpan<char></code>	<code>TokenCounter</code> (подсчёт слов/токенов)	Нет аллокаций подстрок
<code>ArrayPool<T></code> + <code>stackalloc</code>	<code>CompressionCache.ComputeKey</code>	–2 кучи-аллокации на вызов, меньше нагрузки на GC
<code>ValueTask</code>	<code>Operator.DisposeAsync</code>	Освобождение без аллокации <code>Task</code>

4. Примеры `Operator` + MCP-инструменты

Добавлен новый пример `OperatorMcpToolsExample`

(`src/Delibera.ConsoleApp/Examples/OperatorMcpToolsExample.cs`), демонстрирующий работу роли `Operator` с реальными MCP-серверами.

Подключаемые MCP-серверы

Сервер	Транспорт	Команда запуска	Что даёт
 browser	stdio	<code>npx -y @playwright/mcp@latest --headless</code>	Навигация по сайтам, чтение страниц, клики, скриншоты
 marp	stdio	<code>npx -y @marp-team/marp-cli --server <dir></code>	Генерация презентаций (HTML/PDF/PPTX) из Mark-down

Также в примере показан (закомментирован) HTTP/SSE-вариант через `McpServerConfig.Http(...)` для удалённых MCP-серверов с заголовками авторизации.

Что демонстрирует пример

Раздел А — прямой вызов Operator (без совета):

- Ручное создание MCP-клиентов (`McpClientAdapter`) и объекта `Operator` .
- `await using` для корректного освобождения через `ValueTask DisposeAsync` .
- `InitializeAsync()` — подключение к серверам и обнаружение инструментов.
- Две задачи через `ExecuteTaskAsync` :
 1. **Браузер**: открыть `https://modelcontextprotocol.io` и кратко пересказать, что такое MCP.
 2. **Marp**: сгенерировать презентацию из 3 слайдов и сохранить как `deck.html` .

Раздел В — Operator внутри совета (делегирование маркером `[[OPERATOR: ...]]`):

- Построение совета через `CouncilBuilder` с участниками, председателем (`Chairman`) и `Operator`’ом (удобная перегрузка `WithOperator(modelName, provider, servers)`).
- Подписка на `OnRoundCompleted` для вывода активности `Operator` по раундам.
- Сохранение Markdown-отчёта дебатов и готовых презентаций.

Запуск примера

```
# из каталога проекта
dotnet run --project src/Delibera.ConsoleApp -- --operator-mcp
```

Предварительные требования

- **Node.js + npx** — нужны для запуска npx-MCP-серверов (`@playwright/mcp` , `@marp-team/marp-cli`).
- Для браузера при первом запуске Playwright скачает движки: `npx playwright install` .
- **Ollama** запущен локально (`ollama serve`) с моделями `llama3.2` , `qwen2.5` (модель `Operator` можно заменить на любую «дешёвую»).

Пример устойчив к отсутствию окружения: при ошибке выводятся понятные подсказки на русском, процесс не падает аварийно.

5. Сборка и запуск

```
# Сборка обоих проектов
dotnet build

# Сборка в Release
dotnet build -c Release

# Запуск консольного приложения
dotnet run --project src/Delibera.ConsoleApp

# Доступные демонстрации (флаги):
dotnet run --project src/Delibera.ConsoleApp -- --operator           # базовый пример Operator
dotnet run --project src/Delibera.ConsoleApp -- --operator-mcp      #  браузер + Marp
dotnet run --project src/Delibera.ConsoleApp -- --compression        # сжатие контекста
dotnet run --project src/Delibera.ConsoleApp -- --rag                 # RAG
dotnet run --project src/Delibera.ConsoleApp -- --di                 # Dependency Injection
```

Результат сборки:

```
Build succeeded.
    0 Warning(s)
    0 Error(s)
```

Примечание о localhost. В примерах используется адрес `http://localhost:11434` (Ollama) и локальные пути для вывода презентаций. Это localhost той машины, где запускается приложение (среда агента Abacus AI), а не вашего компьютера. Чтобы запустить локально, скачайте файлы через иконку «Files», перейдите в скачанную папку, разверните приложение на своей системе и запустите его.

6. Краткая сводка изменённых файлов

Файл	Изменение
src/Delibera.Core/Delibera.Core.csproj	LangVersion=preview , SourceLink → 10.0.300
src/Delibera.ConsoleApp/Delibera.ConsoleApp.csproj	LangVersion=preview
src/Delibera.Core/Compression/SemanticCompressor.cs	SIMD-векторизация CosineSimilarity + ReadOnlySpan<float>
src/Delibera.Core/Compression/TokenCounter.cs	Перегрузка ReadOnlySpan<char> , span-подсчёт слов
src/Delibera.Core/Compression/CompressionCache.cs	ArrayPool<T> + stackalloc в ComputeKey
src/Delibera.ConsoleApp/Examples/OperatorMcpToolsExample.cs	 Пример Operator с браузером и Marp
src/Delibera.ConsoleApp/Program.cs	Флаг --operator-mcp , исправлено nullable-предупреждение

Документ подготовлен в рамках перевода Delibera на .NET 10 / C# 15.